

Customer

Customer

1. Feature Overview

Staff CRUD for bank customers, search/pagination, detail view, and optional first-account opening on create. Detail page can open additional Current (policy-aware) and Loan-type accounts.

Status: Implemented (Loan product still partial)

2. Business Purpose

Maintain the customer master that owns accounts; support onboarding with an initial product.

3. User Workflow

1. Sidebar → Customers (/app/customers) — ADMIN, EMPLOYEE, MANAGER
2. Search / page / open detail
3. Create customer (dialog) — optionally choose account type + opening deposit
4. Edit customer
5. From detail: Add Current Account (policy dialog) / Add Loan Account
6. Delete — ADMIN only

4. Execution Flow

See [CALL FLOW.md](#).

5. Database Tables

customers (+ creates rows in account when opening products)

6. REST APIs

Method	Path	Roles
GET	/customers	ADMIN, EMPLOYEE, MANAGER
GET	/customers/{id}	ADMIN, EMPLOYEE, MANAGER
POST	/customers	ADMIN, EMPLOYEE
PUT	/customers/{id}	ADMIN, EMPLOYEE
DELETE	/customers/{id}	ADMIN

7. Controllers

com.bankone.customer.controller.CustomerController

8. Services

CustomerService / CustomerServiceImpl

Methods: createCustomer, getCustomerById, getAllCustomers, searchCustomers, updateCustomer, deleteCustomer

Depends on AccountService for optional account on create.

9. Repositories

CustomerRepository extends JpaRepository + JpaSpecificationExecutor

10. DTOs

- CreateCustomerRequest
- UpdateCustomerRequest (@Valid on controller)

API currently returns Customer entity (not a response DTO).

11. Entities

com.bankone.customer.entity.Customer

JSON customerCode via BusinessIdFormatter

12. Utility Classes

- CustomerSpecification.containsText
- com.bankone.common.util.BusinessIdFormatter

13. Configuration Classes

Role matchers in SecurityConfig for /customers/**

14. Validation Rules

Update: @NotBlank, @Email, phone \d{10}, status ACTIVE | INACTIVE | SUSPENDED

Entity-level constraints on persist

Create: service validates before save; account opening reuses account policy rules

15. Security Rules

GET: ADMIN/EMPLOYEE/MANAGER · write: ADMIN/EMPLOYEE · DELETE: ADMIN

Frontend routes mirror roles.

16. Exception Handling

Missing customer → not-found style business exception via global handler.

17. Logging

Hibernate SQL in dev; no dedicated customer audit log.

18. Audit Events

created_at / updated_at on entity only.

19. Testing Strategy

- Create customer without account
- Create with SAVINGS/CURRENT + valid opening deposit
- Search by name/email/phone
- EMPLOYEE cannot DELETE (403)

20. Future Extension Guide

- Introduce CustomerResponse DTO (stop exposing entity)
- Soft-delete instead of hard delete
- KYC documents module
- Wire Loan through real loan domain + policy seed

Future Modification Guide

Requirement: Change phone validation to allow country codes

Item	Detail
Files	UpdateCustomerRequest.java, Customer.java, create request if validated, Angular forms
Classes	UpdateCustomerRequest, Customer
Methods	Field annotations / setters used by updateCustomer
Impact	DB column length phone_number; unique constraint; UI masks

Requirement: Stop opening account during customer create

Item	Detail
Files	CustomerServiceImpl.java, create-customer dialog
Methods	createCustomer() — remove/guard AccountService.openAccount call
Impact	Onboarding UX; Account module unchanged

Requirement: Change customer code format

Item	Detail
Files	BusinessIdFormatter.java, Customer.getCustomerCode()

Item	Detail
------	--------

Impact All UI labels; not a DB migration	
---	--

Call hierarchy (create with account)

```
CustomerController.createCustomer()  
    ↓  
CustomerServiceImpl.createCustomer()  
    ↓  
CustomerRepository.save()  
    ↓  
AccountService.openAccount()  
    ↓  
AccountPolicy validation + AccountRepository.save()
```